Think Bayes

Bayesian Statistics Made Simple

Version 1.0.9

Think Bayes

Bayesian Statistics Made Simple

Version 1.0.9

Allen B. Downey

Green Tea Press

Needham, Massachusetts

Copyright © 2012 Allen B. Downey.

Green Tea Press
9 Washburn Ave
Needham MA 02492

Permission is granted to copy, distribute, and/or modify this document under the terms of the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License, which is available at http://creativecommons.org/licenses/by-nc-sa/4.0/.

The LaTeX source for this book is available from http://greenteapress.com/thinkbayes.

Preface

0.1 My theory, which is mine

The premise of this book, and the other books in the *Think X* series, is that if you know how to program, you can use that skill to learn other topics.

Most books on Bayesian statistics use mathematical notation and present ideas in terms of mathematical concepts like calculus. This book uses Python code instead of math, and discrete approximations instead of continuous mathematics. As a result, what would be an integral in a math book becomes a summation, and most operations on probability distributions are simple loops.

I think this presentation is easier to understand, at least for people with programming skills. It is also more general, because when we make modeling decisions, we can choose the most appropriate model without worrying too much about whether the model lends itself to conventional analysis.

Also, it provides a smooth development path from simple examples to real-world problems. Chapter 3 is a good example. It starts with a simple example involving dice, one of the staples of basic probability. From there it proceeds in small steps to the locomotive problem, which I borrowed from Mosteller's *Fifty Challenging Problems in Probability with Solutions*, and from there to the German tank problem, a famously successful application of Bayesian methods during World War II.

0.2 Modeling and approximation

Most chapters in this book are motivated by a real-world problem, so they involve some degree of modeling. Before we can apply Bayesian methods (or any other analysis), we have to make decisions about which parts of the real-world system to include in the model and which details we can abstract away.

For example, in Chapter 7, the motivating problem is to predict the winner of a hockey game. I model goal-scoring as a Poisson process, which implies that a goal is equally likely at any point in the game. That is not exactly true, but it is probably a good enough model for most purposes.

In Chapter 12 the motivating problem is interpreting SAT scores (the SAT is a standardized test used for college admissions in the United States). I start with a simple model that assumes that all SAT questions are equally difficult, but in fact the designers of the SAT deliberately include some questions that are relatively easy and some that are relatively hard. I present a second model that accounts for this aspect of the design, and show that it doesn't have a big effect on the results after all.

I think it is important to include modeling as an explicit part of problem solving because it reminds us to think about modeling errors (that is, errors due to simplifications and assumptions of the model).

Many of the methods in this book are based on discrete distributions, which makes some people worry about numerical errors. But for real-world problems, numerical errors are almost always smaller than modeling errors.

Furthermore, the discrete approach often allows better modeling decisions, and I would rather have an approximate solution to a good model than an exact solution to a bad model.

On the other hand, continuous methods sometimes yield performance advantages—for example by replacing a linear- or quadratic-time computation with a constant-time solution.

So I recommend a general process with these steps:

1. While you are exploring a problem, start with simple models and implement them in code that is clear, readable, and demonstrably correct. Focus your attention on good modeling decisions, not optimization.

2. Once you have a simple model working, identify the biggest sources of error. You might need to increase the number of values in a discrete approximation, or increase the number of iterations in a Monte Carlo simulation, or add details to the model.

3. If the performance of your solution is good enough for your application, you might not have to do any optimization. But if you do, there are two approaches to consider. You can review your code and look for optimizations; for example, if you cache previously computed results you might be able to avoid redundant computation. Or you can look for analytic methods that yield computational shortcuts.

One benefit of this process is that Steps 1 and 2 tend to be fast, so you can explore several alternative models before investing heavily in any of them.

Another benefit is that if you get to Step 3, you will be starting with a reference implementation that is likely to be correct, which you can use for regression testing (that is, checking that the optimized code yields the same results, at least approximately).

0.3 Working with the code

The code and sound samples used in this book are available from `https://github.com/AllenDowney/ThinkBayes`. Git is a version control system that allows you to keep track of the files that make up a project. A collection of files under Git's control is called a "repository". GitHub is a hosting service that provides storage for Git repositories and a convenient web interface.

The GitHub homepage for my repository provides several ways to work with the code:

- You can create a copy of my repository on GitHub by pressing the Fork button. If you don't already have a GitHub account, you'll need to create one. After forking, you'll have your own repository on GitHub that you can use to keep track of code you write while working on this book. Then you can clone the repo, which means that you copy the files to your computer.
- Or you could clone my repository. You don't need a GitHub account to do this, but you won't be able to write your changes back to GitHub.
- If you don't want to use Git at all, you can download the files in a Zip file using the button in the lower-right corner of the GitHub page.

The code for the first edition of the book works with Python 2. If you are using Python 3, you might want to use the updated code in `https://github.com/AllenDowney/ThinkBayes2` instead.

I developed this book using Anaconda from Continuum Analytics, which is a free Python distribution that includes all the packages you'll need to run the code (and lots more). I found Anaconda easy to install. By default it does a user-level installation, not system-level, so you don't need administrative privileges. You can download Anaconda from `http://continuum.io/downloads`.

If you don't want to use Anaconda, you will need the following packages:

- NumPy for basic numerical computation, http://www.numpy.org/;
- SciPy for scientific computation, http://www.scipy.org/;
- matplotlib for visualization, http://matplotlib.org/.

Although these are commonly used packages, they are not included with all Python installations, and they can be hard to install in some environments. If you have trouble installing them, I recommend using Anaconda or one of the other Python distributions that include these packages.

Many of the examples in this book use classes and functions defined in `thinkbayes.py`. Some of them also use `thinkplot.py`, which provides wrappers for some of the functions in `pyplot`, which is part of `matplotlib`.

0.4 Code style

Experienced Python programmers will notice that the code in this book does not comply with PEP 8, which is the most common style guide for Python (http://www.python.org/dev/peps/pep-0008/).

Specifically, PEP 8 calls for lowercase function names with underscores between words, `like_this`. In this book and the accompanying code, function and method names begin with a capital letter and use camel case, `LikeThis`.

I broke this rule because I developed some of the code while I was a Visiting Scientist at Google, so I followed the Google style guide, which deviates from PEP 8 in a few places. Once I got used to Google style, I found that I liked it. And at this point, it would be too much trouble to change.

Also on the topic of style, I write "Bayes's theorem" with an *s* after the apostrophe, which is preferred in some style guides and deprecated in others. I don't have a strong preference. I had to choose one, and this is the one I chose.

And finally one typographical note: throughout the book, I use PMF and CDF for the mathematical concept of a probability mass function or cumulative distribution function, and Pmf and Cdf to refer to the Python objects I use to represent them.

0.5 Prerequisites

There are several excellent modules for doing Bayesian statistics in Python, including `pymc` and OpenBUGS. I chose not to use them for this book because

you need a fair amount of background knowledge to get started with these modules, and I want to keep the prerequisites minimal. If you know Python and a little bit about probability, you are ready to start this book.

Chapter 1 is about probability and Bayes's theorem; it has no code. Chapter 2 introduces `Pmf`, a thinly disguised Python dictionary I use to represent a probability mass function (PMF). Then Chapter 3 introduces `Suite`, a kind of Pmf that provides a framework for doing Bayesian updates.

In some of the later chapters, I use analytic distributions including the Gaussian (normal) distribution, the exponential and Poisson distributions, and the beta distribution. In Chapter 15 I break out the less-common Dirichlet distribution, but I explain it as I go along. If you are not familiar with these distributions, you can read about them on Wikipedia. You could also read the companion to this book, *Think Stats*, or an introductory statistics book (although I'm afraid most of them take a mathematical approach that is not particularly helpful for practical purposes).

Contributor List

If you have a suggestion or correction, please send email to *downey@allendowney.com*. If I make a change based on your feedback, I will add you to the contributor list (unless you ask to be omitted).

If you include at least part of the sentence the error appears in, that makes it easy for me to search. Page and section numbers are fine, too, but not as easy to work with. Thanks!

- First, I have to acknowledge David MacKay's excellent book, *Information Theory, Inference, and Learning Algorithms*, which is where I first came to understand Bayesian methods. With his permission, I use several problems from his book as examples.
- This book also benefited from my interactions with Sanjoy Mahajan, especially in fall 2012, when I audited his class on Bayesian Inference at Olin College.
- I wrote parts of this book during project nights with the Boston Python User Group, so I would like to thank them for their company and pizza.
- Olivier Yiptong sent several helpful suggestions.
- Yuriy Pasichnyk found several errors.
- Kristopher Overholt sent a long list of corrections and suggestions.
- Max Hailperin suggested a clarification in Chapter 1.

- Markus Dobler pointed out that drawing cookies from a bowl with replacement is an unrealistic scenario.

- In spring 2013, students in my class, Computational Bayesian Statistics, made many helpful corrections and suggestions: Kai Austin, Claire Barnes, Kari Bender, Rachel Boy, Kat Mendoza, Arjun Iyer, Ben Kroop, Nathan Lintz, Kyle McConnaughay, Alec Radford, Brendan Ritter, and Evan Simpson.

- Greg Marra and Matt Aasted helped me clarify the discussion of *The Price is Right* problem.

- Marcus Ogren pointed out that the original statement of the locomotive problem was ambiguous.

- Jasmine Kwityn and Dan Fauxsmith at O'Reilly Media proofread the book and found many opportunities for improvement.

- Linda Pescatore found a typo and made some helpful suggestions.

- Tomasz Miąsko sent many excellent corrections and suggestions.

Other people who spotted typos and small errors include Tom Pollard, Paul A. Giannaros, Jonathan Edwards, George Purkins, Robert Marcus, Ram Limbu, James Lawry, Ben Kahle, Jeffrey Law, and Alvaro Sanchez.